Preserved in the raw record, absent from the export projection

An executable minimal-pair audit of FLARE-AI incident-report converters

State: Complete-study draft, 2026-10-10 UTC. Executed technical case study with separate AI critical review; not externally peer reviewed.

Evidence cutoff: 2026-10-09T23:59:59Z. This covers the public sources and runtime provenance retrieved by that time, not an exhaustive census of available evidence. The implementation examined is commit f51514d4b999fa7988e05d44b9bf18ea0aa98031, dated 2026-08-22. This is a version-specific source audit, not a test of the currently deployed service. Review continued on 2026-10-10 UTC.

AI assistance and review: Codex AI agents performed source research, fixture design, coding, drafting, and separate critical review. A separate reviewer reproduced the results on the same host and dependencies, independently recalculated the case counts, and checked source meanings and claim boundaries. The review and author responses are retained with the study. No human or practitioner validation, external peer review, or institutional endorsement is claimed. Publication quality is a target, not a certification. No project reuse licence has been selected.

Abstract

AI incident reporting increasingly depends on structured forms and export converters. Preserving a report's raw data and preserving the facts exposed through a particular export are different properties. We examine these properties in a pinned public FLARE-AI implementation using 16 source-inspection-guided minimal pairs, comprising 32 wholly synthetic records. Every record passes both the original base Zod schema and the original classification-dependent validator. Under a fixed conversion clock, the complete JSON-LD output distinguishes all 16 pairs and recovers all 32 JSON-safe source records exactly through flare:raw. The JSON-LD projection excluding that raw field distinguishes 6 pairs; the implementation's standalone CERT-style JSON distinguishes 8. These are diagnostic case counts, not estimated loss rates. A concrete counterexample concerns system version: changing an explicit version while holding platform and model fixed produces identical standalone CERT-style output and identical JSON-LD named projections. Conversely, omission and explicit uncertainty remain distinct in the selected severity and prevalence controls. Static inspection shows that the download package also includes an original JSON record; package-level loss was not demonstrated. The contribution is a reproducible, version-specific regression artifact and a distinction between whole-record retention and export-specific accessibility. It does not establish operational harm, recipient-API noncompliance, or novelty of preservation testing in general.

1. Question and governance relevance

An incident report may be collected once and consumed through several exports. A regulator, evaluator, vulnerability coordinator, or internal reviewer may receive only one of those artifacts. The relevant question is therefore specific: **for two valid source records differing in one supplied fact, does a named output artifact still distinguish them, and can its exposed fields answer the corresponding source question?**

This is narrower than asking whether a reporting framework is comprehensive. It is also narrower than assessing a reporting organisation's legal compliance or actual practice. An explicit system version can distinguish the build to investigate; a statement of expected behaviour can distinguish the requirement being evaluated; a coordination instruction can identify a condition for public release. These are

plausible workflow uses, not a practitioner-validated utility model. Our tests concern the supplied assertions, not whether those assertions are true.

The initial candidate study compared OECD and ETSI reporting descriptions. Source review found ample narrative and additional-information channels, so a claim of unavoidable format loss was not warranted. It also found ambiguities in implementing the printed ETSI schema. We therefore chose an authentic executable implementation instead of attributing outcomes from a researcher-invented converter to those standards. The abandoned broader framing and source checks remain in the programme's schema-feasibility notes.

2. Related work and bounded contribution

FLARE-AI already surveys reporting systems and describes semantic conversion and raw-report retention. Its own design supplies the strongest negative control for a loss claim. MIT's October 8, 2026 preliminary practitioner report explicitly calls for testing crosswalks. Holeman and colleagues already test detection-relevant information under security-log normalization. Kumar and colleagues describe versioned agent-incident mappings, unknowns, and provenance. Consequently, neither crosswalks nor decision-relevant preservation tests are new ideas. [1-4]

The distinction between information reconstruction and query preservation is established in database mapping research, and round-trip laws are established in bidirectional transformation work. Our finite test suite is an application of those distinctions. A collision is a witness for the selected inputs and artifact; passing cases are not a proof over all schema-valid records. [5-6]

The bounded contribution is an executable suite for a particular AI incident-reporting implementation, with native-valid minimal pairs, exact source hashes, an explicit output boundary, and retained negative findings. The searched literature did not reveal this exact artifact. That bounded search is not proof of novelty. Full search scope, additional nearby work, unresolved bibliographic discrepancies, and retrieval hashes are preserved in the prior-art gate and manifest.

3. Materials, versions, and policy status

The FLARE project page links to the public implementation repository. We pin its client form schemas, client JSON-LD and CERT-style conversion functions, relevant field definitions, and download hook at one Git commit. Client and server converters are not assumed equivalent. The repository's MIT licence applies to its own code; it does not supply a reuse licence for this research programme. [7]

Material	Version or date	Status and use in this study
FLARE-AI client implementation	Commit f51514d4b999fa7988e05d44b9bf18ea0aa98031; 2026-08-22	Public implementation, executed locally. No claim that it is the live deployed revision.
OECD common reporting framework	Published 2025-02-28	Policy reporting framework; used in design screening, not as universally binding law or an executable JSON schema.
ETSI TS 104 158-2	V1.1.1; publication 2026-03-16	Technical specification; official work-item record says it is not harmonised. Used in feasibility screening, not official conformance testing.
JSON-LD and CERT-style outputs	As generated by the pinned client functions	We test the implementation's artifacts. We do not validate against a recipient's current API or infer statutory notification duties.

The analysis uses no real incident records, user reports, interviews, or private information. Fixture identities and contact addresses are fictional. No record was sent to FLARE, CERT, a regulator, or another recipient. Full third-party articles and restricted standards are retained locally rather than republished. Source pointers and hashes support retrieval; the public implementation is separately available at its immutable revision.

The authentic CERT-style converter inserts a template string saying the report was forwarded to VINCE. That unchanged generated boilerplate is not evidence of transmission: this harness performs no report submission.

4. Diagnostic design and methods

4.1 Source-inspection-guided cases

This is an exploratory diagnostic audit. Source inspection informed the selected fields and the anticipated failure modes. It is not a blind benchmark, a random sample of fields, a preregistered hypothesis test, or an estimate of how users ordinarily populate reports. The fixture file records its creation time and selection rationale. The protocol records the sequence of source inspection, fixture construction, first execution, and subsequent confirmation.

Sixteen pairs vary one declared source path within each pair. The 13 question families concern supplied version, access method, expected behaviour, actual-behaviour statement, harm narrative, prospective severity, prevalence, documented weakness, detection guidance, public-disclosure intent, public-disclosure date, and two native coordination-text locations. Missing and explicit uncertainty are tested only where the actual source supports omission or a suitable value. Disputed and revised states carried in prose are labelled as prose, not as native state enums.

Native UI meaning matters. `severityOfHarm` asks about prospective severity. `detectionMethod` asks how a problem could be detected, not when it was discovered. `mitigationNotes` is presented as embargo or coordination requirements, not containment completion. The harm-state pair keeps an established fictional financial loss constant while changing a separate privacy assessment from a confirmed negative to uninvestigated. This avoids treating a required `real_world_harm=false` boolean as a valid representation of unknown overall harm.

Each record is checked with both `aiFlawReportSchema` and `createAiFlawReportSchema`, passing the record's actual classification flags to the latter. Input bytes are not replaced by schema defaults before conversion. Validation success is not a demonstration that the interactive UI can produce every omitted-field fixture. All attachment arrays are empty; conclusions about raw reconstruction concern JSON-serializable values only.

4.2 Artifact boundaries and controls

We execute the original conversion functions without changing their source bytes. TypeScript execution and module resolution are handled by the documented local harness. The conversion clock is fixed so that incidental generated timestamps cannot distinguish otherwise identical outputs.

Four comparisons are retained:

1. Complete JSON-LD, including `flare:raw`.

2. A research projection of that actual JSON-LD with only `flare:raw` removed. This is an explicit query boundary, not an assertion that the exporter itself deletes the raw record.

3. The `flare:raw` value compared with the complete JSON-safe input record.

4. The actual standalone JSON returned by `convertToCert`.

The full JSON-LD/raw path is the mandatory preservation control. Static inspection of useDownloadReport.ts shows that the package contains JSON-LD, CERT-style JSON, original non-file form JSON, and a separate attachment folder. We did not execute the browser download hook or test attachment fidelity, so this is packaging context rather than an executed package round-trip result.

4.3 Equality, recoverability, and query interpretation

Let x and y be a selected pair with different source answers, and f a fixed deterministic export. If f(x)=f(y), no decoder using that artifact alone can recover both distinct source answers: it receives the same input in both cases. This is a finite collision witness, not a general characterization of the format's expressive capacity.

Equality uses canonical JSON structural comparison. It ignores object-key order but retains string content and array order. We separately inspect the actual target paths listed in mapping.csv. Exact lexical equality is only a diagnostic: capitalization changes such as no to No, or an answer embedded within a longer narrative, do not establish semantic loss. A dedicated string field may itself contain prose. We therefore distinguish whole-artifact collision, named-field observation, raw reconstruction, and narrative carriage.

Missing source fields cannot be silently interpreted as negative findings. A question without an authentic native representation is a collection-coverage limitation, not a conversion failure. The original candidate questions about distinct occurrence/discovery/notification clocks and containment completion are not assigned invented native encodings in this suite.

5. Executed results

The confirmed run contains 32 records and 16 pairs. All records pass both source validators, and the schema-parsed records equal the supplied records. All 32 JSON-LD raw values reconstruct their inputs exactly.

Artifact or projection	Pairs distinguished	Pairs colliding	Interpretation
Complete JSON-LD	16/16	0/16	Every selected difference remains in the complete output.
JSON-LD excluding flare:raw	6/16	10/16	Restricted named projection; raw retention still succeeds.
JSON-LD raw record	16/16	0/16	Exact JSON-safe source recovery for 32/32 records.
Standalone CERT-style JSON	8/16	8/16	Collisions concern this artifact alone.

These denominators count deliberately selected configurations. They must not be read as incident prevalence, production failure probability, completeness percentages, or a ranking of reporting systems.

5.1 Explicit version counterexample

For P01, the platform and model remain fixed while the explicit source version changes from build-2026-08-01 to build-2026-08-02. Both inputs validate. The JSON-LD named aiSystem[0].version contains the model string, and the CERT-style product_version contains platform/model text. Neither changes when only the supplied version changes. The complete standalone CERT-style records collide, as do the JSON-LD named projections. The version distinction remains available in JSON-LD's raw record. [7, converter paths]

P02 separately distinguishes an omitted version from an explicit free-text statement of uncertainty. It yields the same restricted-output collision pattern. This is not a native unknown-state enum; it tests text the source version field accepts.

5.2 Preserved controls and asymmetric exports

The selected severity and prevalence controls preserve missing-versus-`not_sure` distinctions and low/high or rare/common distinctions in both compared exports. The public-disclosure intent pair `no` versus `undecided` remains distinguishable even though a selected target boolean alone would be insufficient: the export's disclosure narrative retains the answer. These are counterexamples to a blanket claim that export conversion collapses uncertainty.

The two export projections also preserve different facts. The selected documented-weakness, disclosure-date, and disclosure-coordination pairs remain distinct in CERT-style text while colliding in the JSON-LD named projection. Detection guidance remains distinct in the JSON-LD named field while colliding in standalone CERT-style output. Such asymmetry makes the artifact boundary part of the claim; a single unqualified completeness score would obscure it.

6. Interpretation, limits, and implications

The strongest positive conclusion is local: selected valid source distinctions are unrecoverable from one pinned standalone artifact, or absent from one explicitly restricted projection. The strongest negative conclusion is equally important: every tested distinction survives in the complete JSON-LD raw record. Static packaging evidence supplies an additional reason not to describe this as general FLARE download data loss.

Retention and usability need separate contracts. A downstream consumer that intentionally consumes only a summary projection should declare that choice and its fallback to raw records. A version-specific regression suite can make that contract inspectable. Adding an envelope containing the input would trivially preserve these JSON-safe records; this study does not claim that such an envelope is a novel governance mechanism. Nor do these tests establish that a particular typed extension improves real response decisions.

The study is limited by diagnostic case selection, one implementation revision, one client code path, a finite set of fields, JSON-only records, and no end-to-end UI or recipient testing. The primary run uses Node 24.14.1, outside the client package's declared Node 20-22 range. A sensitivity run uses Node 22.23.3, within that range: the complete result objects differ only in their two runtime-metadata entries. Every validation result, converter output, pair observation, and source/dependency hash is identical. The separate reviewer reran both configurations and all eight harness tests successfully. Both configurations use the same host and installed dependencies; no supported full-application deployment is demonstrated. Native TypeScript execution and runtime acquisition are documented. The portable Node archive was checked against the official HTTPS SHA-256 manifest, without release-signature verification.

A different exporter revision, source field usage, raw-aware consumer, or package-level workflow could yield a different result. Actual users might repeat dedicated-field facts in their issue description, which could preserve a distinction in additional text; these tests intentionally place a fact in the dedicated field without redundant duplication. That is a permitted input construction, not evidence about ordinary reporting behaviour. No observed harm, legal breach, or operational improvement is inferred.

7. Reproduction and review record

The study README and harness instructions list acquisition and offline execution commands, fixed dependencies, source verification, and independent rerun. data/fixtures.json contains input cases and question definitions; mapping.csv records the source and target interfaces; results/results.json contains individual outputs and validation evidence. The final confirmation run at 2026-10-09T23:55:36.720Z-23:55:37.351Z follows the recorded design freeze. The separate reviewer reproduced the primary result bytes and independently recalculated the 16/16, 16/16, 6/16, and 8/16 distinction counts without importing the harness comparator. The supported-runtime sensitivity was also independently reproduced.

The critical review, execution evidence, and author responses record the actual checks and their limits. Review prompted explicit disclosure of unchanged transmission boilerplate, the supported-runtime sensitivity, and accurate completion-state prose. This was separate AI review on the same machine, not human review or a fresh-environment replication.

The programme also provides a separately tested local snapshot utility. Its tests verify capture bookkeeping and corruption detection, not the truth of source documents or the scientific validity of this study. Restricted snapshots remain outside the public repository.

References

1. Longpre et al. (2026). *FLARE-AI: Flaw Reporting for AI*. arXiv v1, 2026-06-30; PMLR 306. See semantic conversion and Appendix C.4 raw retention.

2. MIT AI Risk Initiative (2026-10-08). *Preliminary findings on how AI incident databases are used*, §3b. Preliminary practitioner findings.

3. Holeman, Hastings and Vaidyan (2026-05-07). *Beyond Collection: Measuring the Detection Efficacy of Modern Security Logging Standards*, arXiv v1, §III-B. Preprint.

4. Kumar et al. (2026-09-10). *The Agent Incident Registry: Toward Preventing Repeated AI Agent Failures*, arXiv v1, §§3.3, 5-6 and Appendix A.3. Preprint.

5. Sequeda, Arenas and Miranker (2012-02-16). *On Directly Mapping Relational Databases to RDF and OWL (Extended Version)*, §3.1.

6. Dončević, Brčić and Mlinarić (v2, 2025-03-06). *Bidirectionalization For The Common People*, §§III-B and IV.

7. FLARE-AI implementation (2026-08-22). Pinned repository. Relevant paths: client/src/entities/ai-flaw-report/model/schema.ts, lib/jsonld-converter.ts, lib/cert-converter.ts, and model/hooks/useDownloadReport.ts; full path prefixes and SHA-256 hashes are in the execution metadata.

8. OECD (2025-02-28). *Towards a common reporting framework for AI incidents*, Artificial Intelligence Papers No. 34. Background feasibility source.

9. ETSI (2026-03-16). *TS 104 158-2 V1.1.1*; official work item. Background feasibility source.